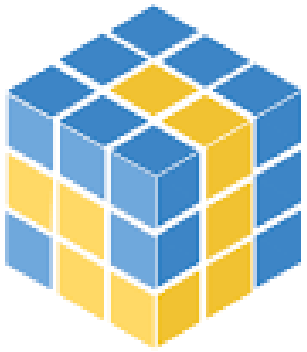


NumPy



NUMPY FROM SCRATCH

1.**Introduction**
2. Getting Started
3. Numpy Array
4. Array Reshaping
5. Arithmetic Operations
6. Indexing and Slicing
7. Broadcasting
8. Universal Functions(uFuncs)
9. Sorting and Searching
10. Stacking and Splitting Arrays
11. Numpy Random Module
12. Linear Algebra with Numpy
13.Quick Cheat Sheet
Final Notes
    Final Verdict

▼ 1.Introduction

- **What is Numpy?**

NumPy (Numerical Python) is a Python library used for fast and efficient numerical computations with arrays.

- **Why use Numpy?**

NumPy is a Python library that lets you efficiently store, manipulate, and perform fast mathematical operations on large arrays of numbers.

- **Who's it for?**

NumPy is for anyone who works with numerical data in Python, including:

- **Data scientists** – analyzing datasets, computing statistics, preparing data for machine learning
- **Machine learning / AI engineers** – handling tensors, matrices, and input data for models
- **Scientists & researchers** – performing simulations, experiments

- **Installing NumPy**

If you have Anaconda/Jupyter:

NumPy comes pre-installed with Anaconda/Jupyter.

Just type this at the top of your notebook:

```
import numpy as np
```

You're ready to go.

If you don't have Anaconda:

Open your terminal or command prompt and run:
`pip install numpy`

Then import it the same way:

```
import numpy as np
```

What does `as np` mean? It's just a shortcut.

- Verify it's installed:

And to verify its installed you can put this in your script in your jupyter notebook or your python script ,we'd look in to that later but for now :

```
import numpy as np
print(np.__version__)

#Result example:2.4.3
```

this will print out a result of the version you are currently using ,these may not be the

---

▼ 2. Getting Started

- **Importing NumPy**:

> I know i have added this before but it is really important so please before you can use NumPy, you need to import it into your Python script. This is the very first thing you do to get started.

```
import numpy as np
```

That is it.

The `np` part is just a shortcut or well said an alias so that instead of typing `numpy` every time, you just type `np`..

`as np` is a standard alias used to shorten `numpy` when writing code.

- **Your First NumPy Array**:

An array is just a list of numbers that NumPy can work with. Here is how you create one:

```
import numpy as np
my_array = np.array([10, 20, 30, 40, 50])
print(my_array)

Output:
[10 20 30 40 50]
```

This is your first numpy array ,and to reflect more on that we would look at types of arrays ,1D array and 2D array and 3D array..

```
import numpy as np
a = np.array([10, 20, 30])          # 1D array
```

```
b = np.array([[1, 2], [3, 4]])      # 2D array
c = np.array([[[1,  2,  3],         # 3D  array (2, 2,
3)
               [4,  5,  6]],
              [[7,  8,  9],
               [10, 11, 12]]])
```

- Use case: Setting up a NumPy environment for a data analysis project

When starting a data analysis project, NumPy is used to set up a fast and efficient environment for working with numerical data.

After installing and importing Numpy , as shown earlier ,then you load or create your dataset as a NumPy array this way :

```
data = np.array([10, 20, 30, 40, 50])
```

And once the data is in an array, you can quickly perform operations like this:

```
print(data.mean())   # average
print(data.sum())    # total
print(data.shape)    # structure of the data
```

▼ 3. Numpy Array

▼ Creating Arrays :

Numpy arrays can be created in several ways :

- **From a List**:

```
import numpy as np
arr1 = np.array([1, 2, 3, 4])
```

- **2D array (matrix)**:

```
import numpy as np
```

```
arr2 = np.array([[1, 2], [3, 4]])
```

- **3D array**

```
import numpy as np
a=np.array([[[1,2,3],[1,2,3],[4,5,6]]])
a.ndim   #result is 3
```

- **Range of numbers** :

`arange` is a NumPy function used to generate a **sequence of numbers**, usually for loops.

```
import numpy as np
arr =np.arange(1,10,2) #start #stop #step
arr      #result: is array([1, 3, 5, 7, 9])
```

- **Linspace**:

`linspace` is a NumPy function used to create **evenly spaced numbers over a specified range (creates evenly spaced values between two points).**

```
import numpy as np
arr=np.linspace(0,1,5) #start #stop #numbers to genera
te
arr      #result:  array([0.  , 0.25, 0.5 , 0.75, 1.
])
```

- **Logspace**:

`logspace` is a NumPy function used to **creates values spaced on a logarithmic scale**.

```
import numpy as np
arr =np.logspace(1,3,3) #logarithmic scale array  -> 1
0^1 -> 10^3 .... 3 points
arr    #result: array([  10.,  100., 1000.])
```

- **Zeros**:

  `zeros` is a NumPy function used to create an array filled with **0s**.

```
import numpy as np
arr=np.zeros([2,3])   #rows,columns   and array full o
f zeros
arr                 #Result :array([[0., 0., 0.], [0.,
0., 0.]])
                    OR
arr =np.zeros(4)      #1D array
arr                   #Result :array([0., 0., 0., 0.])
```

- **Ones** :

`ones` is a NumPy function used to create an array filled with **1s**.

```
import numpy as np
arr = np.ones((2, 3))
arr   #result :array([[1., 1., 1.],[1., 1., 1.]])
```

- **Full**:

`full` is a NumPy function used to create an array of a **specified shape filled with any value you choose**. It's more general than `zeros` or `ones`.

```
import numpy as np
arr=np.full(10,4)   #array full of any value
arr                 #Result:array([4, 4, 4, 4, 4, 4,
4, 4, 4, 4])

                       ALSO
arr=np.full([2,4],7)   #two rows four columns of seven
[rows,column],default value
arr                    #array([[7, 7, 7, 7],[7, 7, 7,
7]])
```

▼ Array data types & Type casting:

So quick one ,ill be showing something and why this it is the case as related to data types so hold on to it :

```
import numpy as np
arr =np.array([1,2,3.1,4,5])
```

Another example:

```
import numpy as np
arr =np.array([1.1,2.1,3.1,4.1,5])
arr     #array([1.1, 2.1, 3.1, 4.1, 5. ])
```

YET Another example:

```
import numpy as np
arr=np.array(["book",1,2,3,4.6])
arr  #array(['book', '1', '2', '3', '4.6'], dtype='<U3
2')
```

COMMON DATA TYPES

DATA TYPE	DESCRIPTION	EXAMPLE VALUES
np.int32	32-bit integer (whole numbers)	-2, 0, 45, 100000
np.int64	64-bit integer (larger range)	-5000000000, 42, 900000000000
np.float32	32-bit floating point (decimals, less precision)	3.14, -0.5, 2.7
np.float64	64-bit floating point (more precise, default)	3.1415926535, -0.123456789
np.bool_	Boolean (true/false)	True, False
np.str_	String (text data)	"hello", "data”
np.object_	Mixed or complex Python objects	[1, "text", 3.5]

You can as well change the data type of an array

```
import numpy as np

arr =np.array([1,2.4,3.1,4,5],dtype=np.int64)
arr.dtype
```

Type casting

So yes in **NumPy**, **type casting** (also called **data type conversion**) is the process of changing an array from one data type to another. Type casting may lead to data loss (e.g., converting float to int removes decimals).

- Explicit Type Casting:

```
import numpy as np

arr = np.array([1, 2.4, 3.1, 4, 5])
print(arr.dtype)  # float64 (default because of 2.4 an
d 3.1)

# Convert to integers
arr_int = arr.astype(np.int64)
print(arr_int)    # [1 2 3 4 5]
print(arr_int.dtype)  # int64

# Convert to float32
arrfl32 = arr.astype(np.float32)
print(arrfl32)    # [1. 2.4 3.1 4. 5.]
print(arrfl32.dtype)  # float32
```

- Implicit Type Casting:

```
arr1 = np.array([1, 2, 3], dtype=int)
arr2 = np.array([1.5, 2.5, 3.5], dtype=float)

result = arr1 + arr2
print(result)       # [2.5 4.5 6.5]
print(result.dtype) # float64
```

Type Casting Error:

```
arr= np.array(["1","2","hello"])  #cannot convert a st
ring into an integer or float
arr2=arr.astype(np.int64)
arr2
```

▼ Array Attributes:

- Shape

This is how to check the dimensions of the array ,returns a **tuple** representing the size along each axis.

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr.shape)                          #(2, 3)    2
rows, 3 columns
```

- Size.

This gives the total number of elements and gives the product (*) of the dimensions

```
import numpy as np
print(arr.size)      6   # 2*3
```

- ndim

This gives the number of dimensions

```
print(arr.ndim)   2  # 2D array
```

- dtype

This tells you the data type of an element and what the array elements are stored as.

```
arr = np.array([1, 2, 3], dtype=np.float32)
print(arr.dtype)                                #float32
```

▼ 4. Array Reshaping

In NumPy, **array reshaping** is about changing the shape (dimensions) of an array without altering its data. NumPy provides three main ways to do this: `reshape`, `ravel`, and `flatten`.

- Reshape:

`reshape` changes the shape of an array without changing the data and also returns a view (no new memory) when possible and you **must provide a shape compatible** with the number of elements.

```
import numpy as np

arr = np.array([[1, 2], [3, 4], [5, 6]])
# shape = (3,2) → total elements = 6

reshaped = arr.reshape(2, 3)  # new shape (2,3)
print(reshaped)
# [[1 2 3]
#  [4 5 6]]
```

- Ravel:

`ravel` flattens the array into a **1D array, r**eturns a **view whenever possible** (changes to the raveled array may affect the original array).

```
flat_view = arr.ravel()
print(flat_view)
```

```
# [1 2 3 4 5 6]

flat_view[0] = 100
print(arr)
# [100 2 ...] → original array is modified because ravel
often returns a view
```

- Flatten:

`flatten` is used flatten an array into 1D.Always returns a copy, so the original array is not affected.

```
flat_copy = arr.flatten()
flat_copy[0] = 100
print(arr)        # original array unchanged
print(flat_copy)  # first element changed
```

▼ 5. Arithmetic Operations

This section is about doing calculations on Numpy arrays, Operations are vectorized (no loops required).

- **Element-wise operations (`+`, `-`, `*`, `//`, `/`) :**

`+` -Addition

`-` -Subtraction

`*` -Multiplication

`/` -Division

`//` -Floor division

```
a=np.array([1,2,3,4,5,6])
b=np.array([2,3,5,3.1,8,9])
```

```
print(a+b)    #[ 3.   5.   8.   7.1 13.  15. ]
print(a-b)    #[-1.  -1.  -2.   0.9 -3.  -3. ]
print(a/b)    #[0.5        0.66666667 0.6        1.29032
258 0.625      0.66666667]
```

```
print(a * b)   #[ 2.   6.  15.  12.4 40.  54. ]
print(a // b)  #[0. 0. 0. 1. 0. 0.]
```

- **Power and Modulus:**

Power (``) → raises numbers

Modulus (`%`) → gives remainder

```
print(b%a)    #[0.  1.  2.  3.1 3.  3. ]
print(a ** 2)  #[ 1  4  9 16 25 36]
```

Explanation:

- `2^2 = 4`, `3^2 = 9`
- `3 % 2 = 1` (remainder when divided)

- **Aggregations**(`sum` , `mean` , `min` , `max` , `std`):

`sum` – total of all elements

`mean` – average value

`min` – smallest value

`max` – largest value

`std` – standard deviation (measure of spread)

`var` -(square of standard deviation)

`median` -(middle value)

```
arr =np.array([1,2,3])

print(np.sum(arr))          #6
print(np.median(arr))       #2.0
print(np.std(arr))          #0.816496580927726
print(np.min(arr))          #1
```

```
print(np.max(arr))         #3
print(np.var(arr))         #0.6666666666666666
```

▼ 6. Indexing and Slicing

Indexing and slicing let you access or extract specific elements or subarrays from a NumPy array. It's similar to Python lists, but works for multi-dimensional arrays too.NumPy indexing is faster and supports multi-dimensional access.

```
arr = np.array([10, 20, 30, 40, 50, 60])
```

- **1D Array Indexing :**

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

# Access a single element (0-based index)
print(arr[0])  # 10
print(arr[3])  #40
```

- 1D Array Slicing:

```
# Slice: arr[start:stop:step]
print(arr[1:4])   # [20 30 40]  (from index 1 to 3)
print(arr[:3])    # [10 20 30]  (start to index 2)
print(arr[2:])    # [30 40 50]  (index 2 to end)
print(arr[::2])   # [10 30 50]  (every 2nd element)
```

- 2D Array Indexing :

```
arr2d = np.array([[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]])

# Access single element: arr2d[row, column]
```

```
print(arr2d[0, 1])  # 2
print(arr2d[2, 2])  # 9
```

- 2D Array Slicing:

```
# Slice rows
print(arr2d[0:2, :])  # first 2 rows
# [[1 2 3]
#  [4 5 6]]

# Slice columns
print(arr2d[:, 1:3])  # last 2 columns
# [[2 3]
#  [5 6]
#  [8 9]]

# Slice subarray
print(arr2d[1:3, 0:2])
# [[4 5]
#  [7 8]]
```

- Boolean Indexing (Conditional)

```
# Select elements based on condition
print(arr[arr > 30])  # [40 50]
```

- Multidimensional Slicing :

```
import numpy as np

matrix = np.array([[1, 2, 3, 4],
                   [4, 5, 6, 7],
                   [7, 8, 9, 10]])
```

- Modifying values:

You can **change values directly**

```
arr = np.array([10, 20, 30])

arr[1] = 100
print(arr)   # [10 100 30]
```

Also works with slicing:

```
arr[0:2] = 0
print(arr)   # [0 0 30]
```

- **Advanced indexing:**
  - `np.take() :`

  Another way to select elements using indices.

  ```
  arr = np.array([10, 20, 30, 40])

  print(np.take(arr, [1, 3]))   # [20 40]
  ```

  - Indexing with Multiple Arrays (2D specific):

  ```
  arr2d = np.array([[1, 2],
                    [3, 4]])

  print(arr2d[[0, 1], [1, 0]])   # [2 3]
  ```

▼ 7. Broadcasting

Broadcasting is a feature in NumPy that allows arrays of **different shapes** to be used together in arithmetic operations, So literally NumPy **automatically expands** the smaller array to match the shape of the larger one.Broadcasting avoids copying data, making operations memory-efficient.

Rules of Broadcasting :

NumPy compares shapes element by element from the trailing (right-most) dimension:

- If the dimensions are equal, they are compatible.

- If one dimension is 1 ,it is stretched to match the other.
- if dimensions differ and neither is 1,broadcasting fails.

Simple Scalar Broadcasting

```
arr=np.array([1,2,3,4])
arr + 10                #[11,12,13,14]  <------ 10 is br
oadcast across all elements
arr * 2                 #[2,4,6,8]
arr **2                 #[1,4,9,16]
```

1D + Scalar Broadcasting

```
a=np.array([1,2,3])
b=5
print(a + b)        #[6,7,8]

#shape breakdown:
#a   ->(3,)
#b --->()     <----- Scalar Stretched to (3,)
```

2D Array + 1D Array

```
matrix =np.array([[1,2,3],[4,5,6]])
row =np.array ([10,20,30])

matrix + row
#[[11,22,33],[14,25,36]]


#Shape breakdown:
```

```
#matrix -----> (2,3)
#row  ------> (3,)    <------broadcast along axis 0
```

Column Vector Broadcasting

```
matrix =np.array([[1,2,3],[4,5,6]])
col =np.array ([[10],[20]])

matrix + col
#[[11,12,13],[24,25,26]]

#Shape breakdown
#matrix ->(2,3)
#col -> (2,1)   <-----broadcasting along axis 1
```

Broadcasting Failure Example

```
a=np.array([[1,2,3]])   #shape (1,3)
b=np.array([[1,2]])     # shape (1,2)

a + b # ValueError:Operands could not be broadcast
      #3 and 2 are incompatible (neither is 1)
```

▼ 8. Universal Functions(uFuncs)

Universal Functions (ufuncs) are NumPy functions that operate element-wise on arrays without explicit loops. They are implemented in optimized C code, so they are very fast.

Math ufuncs

```
import numpy as np

arr = np.array([1, 4, 9, 16])

print(np.sqrt(arr))            # [1. 2. 3. 4.]
```

```
print(np.exp(arr))             # [2.718... 54.598... 810
3.083... 8886110.52...]
print(np.log(arr))             # natural log:  [0.
1.38629436 2.19722458 2.77258872]
print(np.log10(arr))           # base-10 log:  [0.
0.60205999 0.95424251 1.20411998]
print(np.abs(np.array([-1, -2, 3])))  # [1 2 3]
```

Trigonometric ufuncs

```
import numpy as np

angles = np.array([0, np.pi / 2, np.pi])

print(np.sin(angles))  # [0. 1. 0.] (approx)
print(np.cos(angles))  # [ 1.  0. -1.]
print(np.tan(angles))  # [0. large 0.]
```

Rounding

```
import numpy as np

arr = np.array([1.2, 2.7, 3.5, -1.8])

print(np.floor(arr))  # [ 1.  2.  3. -2.]  (always round
s down)
print(np.ceil(arr))   # [ 2.  3.  4. -1.]  (always round
s up)
print(np.round(arr))  # [ 1.  3.  4. -2.]  (rounds to ne
arest)
```

Comparison ufuncs

```
import numpy as np

a = np.array([1, 5, 3])
b = np.array([2, 4, 3])

print(np.maximum(a, b))  # [2 5 3]  (element-wise max)
print(np.minimum(a, b))  # [1 4 3]  (element-wise min)
```

Methods of ufuncs

Key features

- **Element-wise operations**
- **Broadcasting support**
- **Type casting**
- **Reduce operations**
  - Example:

```
np.add.reduce(arr)   # sums all elements
```

Methods

- `reduce()` → reduces an array to a single value
- `accumulate()` → cumulative results
- `outer()` → pairwise operations
- `at()` → in-place operations
  - Example:

```
np.add.accumulate(arr)  # cumulative sum
```

Example

```
import numpy as np

arr1 = np.array([1, 2, 3])
```

```
arr2 = np.array([4, 5, 6])

result = np.add(arr1, arr2)
print(result)  # [5 7 9]
```

Key takeaway:

Ufuncs are **fast, vectorized operations on arrays** that work **without loops**.

▼ 9. Sorting and Searching

▼ Sorting

Sorting means arranging values in ascending (default) or descending order.

- `np.sort()` returns a **new sorted copy** and leaves the original array unchanged.
- `.sort()` sorts the **original array in-place** and returns `None`, modifies the original array and does not return a new array.

```
import numpy as np

arr = np.array([3, 1, 4, 1, 5, 9, 2, 6])

sorted_arr = np.sort(arr)
print(sorted_arr)  # [1 1 2 3 4 5 6 9]
print(arr)         # [3 1 4 1 5 9 2 6]  (unchanged)

arr.sort()
print(arr)         # [1 1 2 3 4 5 6 9]  (changed)
```

▼ Descending sort

A common pattern is to sort ascending, then reverse the result:

```
import numpy as np

arr = np.array([3, 1, 4, 1, 5])

# Descending order
```

```
desc = np.sort(arr)[::-1]
print(desc)  # [5 4 3 1 1]
```

▼ Sorting 2D arrays (axis)

In 2D arrays, `axis` tells NumPy the direction to sort:

- `axis=0` → sort **each column (top to bottom)**
- `axis=1` → sort **each row (left to right)**

```
import numpy as np

mat = np.array([[3, 1],
                [2, 4]])

print(np.sort(mat, axis=0))
# [[2 1]
#  [3 4]]

print(np.sort(mat, axis=1))
# [[1 3]
#  [2 4]]
```

▼ `argsort` (indices that would sort)

`np.argsort()` returns the indices that would sort the array, which can be used to reorder the array.

```
import numpy as np

arr = np.array([30, 10, 20])

idx = np.argsort(arr)
print(idx)         # [1 2 0]
print(arr[idx])    # [10 20 30]  (use indices to get s
orted values)
```

▼ Powerful use-case: sort one array using another

```
import numpy as np

names = np.array(["A", "B", "C"])
scores = np.array([70, 50, 90])

idx = np.argsort(scores)
print(names[idx])   # ['B' 'A' 'C']
```

▼ Searching

Searching means finding where certain values are located (by index).

▼ `np.where` (find indices or choose values)

`np.where` can be used for both **filtering** and **conditional replacement**.

- `np.where(condition)` returns the indices where the condition is `True`.
- `np.where(condition, x, y)` returns an array where each element is `x` if True, otherwise `y`.

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print(np.where(arr > 25))
# (array([2, 3, 4]),)

print(np.where(arr > 25, arr, 0))
# [ 0  0 30 40 50]
```

Real-life example: replace negative values with 0

```
import numpy as np

arr = np.array([-1, 2, -3, 4])
print(np.where(arr < 0, 0, arr))  # [0 2 0 4]
```

▼ `argmin` and `argmax` (index of min or max)

- `np.argmin(arr)` returns the **index** of the smallest value.
- `np.argmax(arr)` returns the **index** of the largest value.

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print(np.argmin(arr))  # 0  (10 is smallest)
print(np.argmax(arr))  # 4  (50 is largest)
```

Important 2D behavior (flattened index vs axis)

```
import numpy as np

mat = np.array([[1, 5],
                [3, 2]])

print(np.argmax(mat))          # 1 (flattened)
print(np.argmax(mat, axis=0))  # [1 0] (per column)
print(np.argmax(mat, axis=1))  # [1 0] (per row)
```

▼ `np.searchsorted` (optional but strong)

Finds the index where a value should be inserted to maintain sorted order.

```
import numpy as np

arr = np.array([10, 20, 30, 40])
print(np.searchsorted(arr, 25))  # 2
```

▼ 10. Stacking and Splitting Arrays

Stacking combines arrays into one larger array.

Splitting divides an array into smaller arrays.

💡 **Key rule:** arrays must have **compatible shapes** for stacking.

▼ Stacking (combining arrays)

```
import numpy as np

# 1D examples
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

np.hstack([a, b])          # [1 2 3 4 5 6]
np.vstack([a, b])          # [[1 2 3],
                           #  [4 5 6]]
np.concatenate([a, b])     # [1 2 3 4 5 6]
```

▼ What is the difference?

- `np.hstack()` joins arrays **along columns** (axis=1 for 2D)
- `np.vstack()` joins arrays **along rows** (axis=0)
- `np.concatenate()` is the general function where you choose the `axis`

▼ Stacking 2D arrays + using `axis`

```
m1 = np.array([[1, 2],
               [3, 4]])

m2 = np.array([[5, 6],
               [7, 8]])

np.hstack([m1, m2])                # [[1 2 5 6],
                                   #  [3 4 7 8]]

np.vstack([m1, m2])                # [[1 2],
                                   #  [3 4],
                                   #  [5 6],
                                   #  [7 8]]
```

```
np.concatenate([m1, m2], axis=0)    # same as vstac
k
np.concatenate([m1, m2], axis=1)    # same as hstac
k
```

▼ `np.stack()` (adds a new dimension)

`np.stack()` is different from `hstack` and `vstack` because it creates a **new axis**.

```
np.stack([a, b], axis=0)   # [[1 2 3],
                           #  [4 5 6]]

np.stack([a, b], axis=1)   # [[1 4],
                           #  [2 5],
                           #  [3 6]]
```

▼ Splitting (dividing arrays)

```
# 1D array
arr = np.array([1, 2, 3, 4, 5, 6])

np.split(arr, 3)          # [array([1, 2]), array([3,
4]), array([5, 6])]
np.split(arr, [2, 4])     # split at indices 2 and 4
                          # -> [array([1, 2]), array
([3, 4]), array([5, 6])]
```

⚠️ **Equal splits must divide evenly**, otherwise you get an error.

```
# arr has 5 elements, so it cannot be split ev
enly into 3 parts
# np.split(np.array([1, 2, 3, 4, 5]), 3)  # Va
lueError
```

▼ 2D splitting: `hsplit` and `vsplit`

- `np.hsplit()` splits **columns**
- `np.vsplit()` splits **rows**

```
mat = np.array([[1, 2, 3, 4],
                [5, 6, 7, 8]])

print(mat.shape)  # (2, 4)

np.hsplit(mat, 2)  # two arrays of shape (2, 2)  [a
rray([[1, 2], [5, 6]]),array([[3, 4],[7, 8]])]


np.vsplit(mat, 2)  # two arrays of shape (1, 4)  [
array([[1, 2, 3, 4]]),array([[5, 6, 7, 8]])]
```

▼ 11. Numpy Random Module

The `numpy.random` module is used to generate random numbers, arrays, and samples from probability distributions.

- **Setting a Seed:** To ensure your results are the same every time your code runs, use a random seed.

```
import numpy as np
rng = np.random.default_rng(42)   # recommended modern a
pproach
```

- **Common Random Functions:**
  - Uniform Distribution (0 to 1):

```
rng.random(3)        # 1D array
rng.random((2, 3))   # 2×3 matrix
```

- Standard Normal Distribution:

```
rng.standard_normal(4)
```

- Random Integers:

```
rng.integers(1, 10, 5)
```

- Random Selection:

```
rng.choice([10, 20, 30, 40], size=3)
```

- Shuffling Arrays:

```
arr = np.array([1, 2, 3, 4, 5])

rng.shuffle(arr)      # modifies original array
print(arr)
```

- Shuffle vs Permutation

```
rng.permutation(arr)  # returns a new shuffled array
```

- Probability Distributions:
  - Normal Distribution:

```
rng.normal(loc=0, scale=1, size=5)
```

  - Uniform Distribution:

```
rng.uniform(low=0, high=10, size=5)
```

  - Binomial Distribution

```
rng.binomial(n=10, p=0.5, size=5)
```

Old way	New way
`np.random.rand()`	`rng.random()`
`np.random.randn()`	`rng.standard_normal()`
`np.random.randint()`	`rng.integers()`
`np.random.choice()`	`rng.choice()`

▼ 12. Linear Algebra with Numpy

NumPy provides a powerful linear algebra module, `np.linalg`, used for matrix operations in machine learning, data science, and engineering.

- **Matrix Multiplication:**

Matrix multiplication combines two arrays based on linear algebra rules (not element-wise).

```
import numpy as np

A = np.array([[1, 2],
              [3, 4]])

B = np.array([[5, 6],
              [7, 8]])

np.dot(A, B)
# [[19 22]
#  [43 50]]

A @ B   # preferred modern syntax
```

- **Transpose**

```
np.linalg.inv(A)
# [[-2.   1. ]
```

```
#  [ 1.5 -0.5]]
```

The transpose flips rows into columns.

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])

A.T
# [[1 4]
#  [2 5]
#  [3 6]]
```

- **Determinant and Inverse**

  Determinant

  The determinant is a scalar value that describes properties of a matrix (e.g., invertibility).

  ```
  A = np.array([[1, 2],
                [3, 4]])

  np.linalg.det(A)   # -2.0
  ```

  Inverse

  The inverse of a matrix is used to solve linear equations.

- **Eigenvalues and Eigenvectors**

  Eigenvalues and eigenvectors describe how a matrix transforms space.

  ```
  A = np.array([[4, 2],
                [1, 3]])

  eigenvalues, eigenvectors = np.linalg.eig(A)

  print(eigenvalues)    # [5. 2.]
  print(eigenvectors)   # columns are eigenvectors
  ```

Solving a Linear System

Solve the equation: Ax=b

```
# Solving a system of equations:
# 2x + y = 8
# x + 3y = 13

A = np.array([[2, 1],
              [1, 3]])

b = np.array([8, 13])

np.linalg.solve(A, b)  # [3. 2.]
```

Key Notes

- Matrix multiplication is **not element-wise** (use for element-wise)
- Use `@` for matrix multiplication
- `np.linalg` functions require **compatible shapes**
- Not all matrices have an inverse (determinant must not be zero)
- Prefer `np.linalg.solve()` over `inv(A) @ b`

▼ 13.Quick Cheat Sheet

Category	Task	Code
Setup	Import NumPy	`import numpy as np`
Arrays	Create 1D array	`np.array([1, 2, 3])`
	Create 2D array	`np.array([[1, 2], [3, 4]])`
	Zeros array	`np.zeros((2, 3))`
	Ones array	`np.ones((2, 3))`
	Range array	`np.arange(0, 10, 2)`
	Evenly spaced	`np.linspace(0, 1, 5)`

Category	Task	Code
Array Info	Shape	`arr.shape`
	Size	`arr.size`
	Dimensions	`arr.ndim`
	Data type	`arr.dtype`
Reshaping	Reshape	`arr.reshape(2, 3)`
	Flatten (copy)	`arr.flatten()`
	Flatten (view)	`arr.ravel()`
	Transpose	`arr.T`
Math Operations	Sum / Mean / Std	`np.sum(arr), np.mean(arr), np.std(arr)`
	Min / Max	`np.min(arr), np.max(arr)`
	Index of Min / Max	`np.argmin(arr), np.argmax(arr)`
Sorting & Filtering	Sort	`np.sort(arr)`
	Argsort (indices)	`np.argsort(arr)`
	Where condition	`np.where(arr > 5)`
	Boolean filtering	`arr[arr > 5]`
Stacking & Joining	Vertical stack	`np.vstack([a, b])`
	Horizontal stack	`np.hstack([a, b])`
	Concatenate	`np.concatenate([a, b])`
Type Handling	Type casting	`arr.astype(np.float32)`
Linear Algebra	Matrix multiply	`A @ B`
	Inverse	`np.linalg.inv(A)`
	Determinant	`np.linalg.det(A)`
	Solve Ax = b	`np.linalg.solve(A, b)`
Random (Modern API)	Create generator	`rng = np.random.default_rng(42)`
	Random floats [0,1)	`rng.random(3)`
	Random integers	`rng.integers(0, 10, 5)`
	Random choice	`rng.choice([1,2,3], size=2)`
	Shuffle (in-place)	`rng.shuffle(arr)`
	Permutation (copy)	`rng.permutation(arr)`

Category	Task	Code
Distributions	Normal distribution	`rng.normal(0, 1, size=5)`
	Uniform distribution	`rng.uniform(0, 10, size=5)`
	Binomial distribution	`rng.binomial(n=10, p=0.5, size=5)`

Final Notes

- Uses **modern NumPy practices** (`default_rng`)
- Covers **core + intermediate + important advanced concepts**
- Structured for **quick revision and real use**

Final Verdict

NumPy is a fundamental library for numerical computing in Python and serves as the backbone for data science, machine learning, and scientific computing workflows.

Through this guide, I have learned how to:

- Create and manipulate arrays efficiently
- Perform fast vectorized operations without loops
- Reshape and transform data structures
- Apply broadcasting for operations across different shapes
- Use powerful tools like sorting, searching, stacking, and splitting
- Generate random data and work with probability distributions
- Apply linear algebra concepts such as matrix multiplication, inversion, and solving systems of equations

One key takeaway is that **NumPy is not just about arrays**, but about writing **efficient, scalable, and optimized code** for real-world data problems.

I also understand the importance of:

- Using vectorized operations instead of loops
- Writing memory-efficient code

- Leveraging NumPy as a foundation for libraries like Pandas, TensorFlow, and PyTorch

Going forward, the next step is to:

- Apply NumPy in real datasets (Pandas)
- Use it in machine learning workflows
- Build small projects that reinforce these concepts

In summary:

NumPy is an essential tool that transforms Python into a high-performance environment for numerical computation, and mastering it is a critical step toward becoming a skilled data scientist or AI engineer.